Computer Language Processing - CheatSheet

IN BA6 - Viktor Kuncak

Notes by Ali EL AZDI

March 23th, 2026

DFA. $A = \langle \Sigma, Q, q_0, \delta, F \rangle$ where:
Σ: alphabet Q: finite set of states $q_0 \in Q$: start state
$\delta : Q \times \Sigma \to Q$: transition function $F \subseteq Q$: accepting states
Accepts w if unique path $q_0 \xrightarrow{w} q_f \in F$.
NFA. $A = \langle \Sigma, Q, q_0, \delta, F \rangle$ where:
$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$: transition function (allows ε-transitions).
Accepts w if $\exists$ path $q_0 \xrightarrow{w} q_f \in F$.

Determinization. NFA → DFA via subset construction where $Q_{\text{DFA}} \subseteq 2^{Q_{\text{NFA}}}$.
Running NFA. Track state set $S_0 = \{q_0\}$, $S_{i+1} = \bigcup_{q \in S_i} \delta(q, w_{i+1})$. Accept $\iff S_n \cap F \neq \emptyset$.
Regex ≡ Automata. L is regular $\iff \exists$ regex e s.t. $L(e) = L \iff \exists$ FA $\mathscr{A}$ s.t. $L(\mathscr{A}) = L$.
Derivatives ≡ Automata. States represent residual languages: $q \approx \partial_w L$.
Limits. FAs have finite memory $\implies$ must loop on long words. Cannot do unbounded counting or nesting (e.g. balanced '()', $a^n b^n$).

Pumping Lemma. If L is a regular language, then there exists a positive integer p (the pumping length) such that every string $s \in L$ for which $|s| \geq p$, can be partitioned into three pieces, $s = xyz$, such that:
- $|y| > 0$
- $|xy| \leq p$
- $\forall i \geq 0.\ xy^i z \in L$

Induction on Strings. To prove $P(s)$ for all strings s, prove the base case $P(\varepsilon)$, then assume $P(s')$ (**IH**) to prove $P(x \cdot s')$.
Application to matchR: Reduce $P(r, s)$ using $\text{matchR}(r, s) \iff \text{nullable}(\partial_s(r))$, where:

$$\partial_\varepsilon(r) = r, \qquad \partial_{x \cdot s'}(r) = \partial_{s'}(\partial_x(r)).$$

Base case ($s = \varepsilon$) Evaluate $P(r, \varepsilon)$ using $\text{nullable}(r)$.
Inductive step ($s = x \cdot s'$) Assume $P(r', s')$ (**IH**). Unfold $\text{matchR}(r, x \cdot s') = \text{matchR}(\delta_x(r), s')$.

Language. A set $L \subseteq A^*$ of words over an alphabet A.
Concatenation. $L_1 \cdot L_2 = \{w_1 w_2 \mid w_1 \in L_1,\ w_2 \in L_2\}$.
Exponentiation. $L^0 = \{\varepsilon\}$, $L^{n+1} = L \cdot L^n$, so $L^n = \{w_1 \cdots w_n \mid w_i \in L\}$.
Kleene star. $L^* = \bigcup_{n \geq 0} L^n$, all finite concatenations of words from L, including ε.
Kleene star finiteness. L^* is finite only when $L = \emptyset$ or $L \subseteq \{\varepsilon\}$.
Characteristic function. $f_L : A^* \to \{0, 1\}$ where $f_L(w) = 1$ iff $w \in L$
Common Regex Functions:
nullable(e): true iff $\varepsilon \in L(e)$.
first(e) = $\{a \in A \mid \exists v.\ av \in L(e)\}$: valid first characters in $L(e)$.
prefix(e): regex matching all prefixes of words in $L(e)$.

Regex e	**nullable**(e)	**first**(e)	**prefix**(e)
$\emptyset$	False	$\emptyset$	$\emptyset$
ε	True	$\emptyset$	ε
$a \in A$	False	$\{a\}$	$\varepsilon \mid a$
$[a_1 - a_2]$	False	$\{a \mid a_1 \leq a \leq a_2\}$	$\varepsilon \mid [a_1 - a_2]$
$e_1 \mid e_2$	$\text{nullable}(e_1) \vee \text{nullable}(e_2)$	$\text{first}(e_1) \cup \text{first}(e_2)$	$\text{prefix}(e_1) \mid \text{prefix}(e_2)$
$e_1 e_2$	$\text{nullable}(e_1) \wedge \text{nullable}(e_2)$	$\text{first}(e_1) \cup (\text{first}(e_2)$ if $\text{null}(e_1)$ else $\emptyset)$	$\text{prefix}(e_1) \mid e_1 \text{prefix}(e_2)$
e^*	True	first(e)	$e^* \text{prefix}(e)$
e^+	nullable(e)	first(e)	$e^* \text{prefix}(e)$
$e?$	True	first(e)	prefix(e)
e^n $(n > 0)$	$n = 0 \vee \text{nullable}(e)$	first(e)	$\bigcup_{i=0}^{n-1} e^i \text{prefix}(e)$

Token priority. When the same string matches multiple token classes, the one listed first (highest priority) wins.
Language derivative. $\partial_x L = \{w \in A^* \mid xw \in L\}$: remove leading x from words of L that start with x.

Operation	**Derivative**	**Operation**	**Derivative**
Word $\partial_v L$	$\{w \in A^* \mid vw \in L\}$	**Union** $L_1 \cup L_2$	$\partial_x L_1 \cup \partial_x L_2$
Word chain $\partial_\varepsilon L = L$	$\partial_{av} L = \partial_v(\partial_a L)$	**Intersect** $L_1 \cap L_2$	$\partial_x L_1 \cap \partial_x L_2$
Empty language $\emptyset$	$\emptyset$	**Difference** $L_1 \setminus L_2$	$\partial_x L_1 \setminus \partial_x L_2$
Empty word $\{\varepsilon\}$	$\emptyset$	**Complement** $\overline{L}$	$\overline{\partial_x L}$
Single letter $\{y\}$	$\{\varepsilon\}$ if $x = y$ else $\emptyset$	**Universal** A^*	A^*
Kleene star L^*	$(\partial_x L)L^*$		
Concat $L_1 L_2$	$(\partial_x L_1)L_2 \cup (\partial_x L_2$ if $\varepsilon \in L_1$ else $\emptyset)$		

Language decomposition. Over $A = \{a, b\}$: $L = (a\partial_a L) \cup (b\partial_b L) \cup \{\varepsilon \mid \text{nullable}(L)\}$. A language is split by its possible first letters.
Regex derivative rules. $\partial_a \varepsilon = \emptyset$, $\partial_a \emptyset = \emptyset$. $\partial_a b = \varepsilon$ if $b = a$ else $\emptyset$. $\partial_a(r_1 \mid r_2) = \partial_a r_1 \mid \partial_a r_2$. $\partial_a(r^*) = (\partial_a r) r^*$. $\partial_a(r_1 r_2) = (\partial_a r_1) r_2 \mid \partial_a r_2$ (if $\text{nullable}(r_1)$) else $(\partial_a r_1) r_2$.
Membership test via derivatives. $w \in L \iff \varepsilon \in \partial_w L \iff \text{nullable}(\partial_w L)$. So to test a word, derive successively along its letters, then check nullable.
Example.
1. $r = (a \mid b)^*,\ s = abc$

$$r_0 = (a \mid b)^*$$
$$r_1 = \partial_a(r_0) = (a \mid b)^*$$
$$r_2 = \partial_b(r_1) = (a \mid b)^*$$
$$r_3 = \partial_c(r_2) = \emptyset$$
$$\text{nullable}(r_3) = \text{false} \implies abc \notin L\big((a \mid b)^*\big).$$

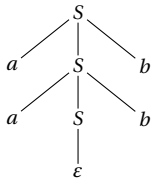
Context-Free Grammar (CFG). A formal way to describe how strings are built. $G = (V, \Sigma, P, S)$ where:
V: variables / nonterminals Σ: terminals (actual symbols in final words)
P: production rules S: start symbol
Constructing CFGs. Detect pattern → define base case (smallest word) + recursive growth rule.
Regex to CFG & Common Patterns.

Pattern	**CFG Rule** (S)	**Pattern**	**CFG Rule** (S)
Union $A \mid B$	$S \to A \mid B$	**Option** $A?$	$S \to A \mid \varepsilon$
Concat AB	$S \to AB$	**Matching** $a^n b^n$	$S \to aSb \mid \varepsilon$
Star A^*	$S \to AS \mid \varepsilon$	**Even Palindr.**	$S \to aSa \mid bSb \mid \varepsilon$
Plus A^+	$S \to AS \mid A$	**Odd Palindr.**	$S \to aSa \mid bSb \mid a \mid b$
Any word Σ^*	$S \to aS \mid bS \mid \varepsilon$	**Balanced** ()	$S \to (S)S \mid \varepsilon$
≥ Match $a^n b^m (n \geq m)$	$S \to aS \mid aSb \mid \varepsilon$	**Nested 3** $a^n b^m c^{n+m}$	$S \to aSc \mid T,\ T \to bTc \mid \varepsilon$
≤ Match $a^n b^m (m \geq n)$	$S \to Sb \mid aSb \mid \varepsilon$	**Nested 3** $a^{n+m} b^n c^m$	$S \to aSc \mid T,\ T \to aTb \mid \varepsilon$
Double $a^n b^{2n}$	$S \to aSbb \mid \varepsilon$	**Equal Counts** $\#a = \#b$	$S \to aSb \mid bSa \mid SS \mid \varepsilon$
Middle a	$S \to aSa \mid aSb \mid bSa \mid bSb \mid a$	**Copying** ww	**NOT CFG!**
Triple $a^n b^n c^n$	**NOT CFG!**	**Prefix** $a^i b^j c^k, i = j + k$	$S \to aSc \mid T,\ T \to aTb \mid \varepsilon$

Example. $L = \{a^n b^m c^{n+m} \mid n, m \geq 0\}$ $\quad S \to aSc \mid T,\ T \to bTc \mid \varepsilon$

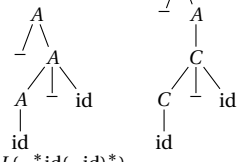
Parse Tree. A visual representation of how a string is derived using a Grammar $G = (A, N, S, R)$. A tree t is a valid parse tree if it has ordered children where:
- **Root** is the start symbol S. **Leaves** are terminal characters (A).
- **Inner nodes** are non-terminal variables (N).
- **Branches** match the rules: if node n has children $p_1 \ldots p_k$ (left-to-right), then $(n \to p_1 \ldots p_k)$ must be a rule in R.

Yield: The final string formed by reading the leaves from left to right.
Language $L(G)$: The set of all possible yields (words) from all valid parse trees.
Note: Checking if t is a valid tree is easy, but given just a word w, finding if a parse tree exists is much harder (Parsing).
Precedence Hints.
Tighter binding = Evaluated First = Deeper in parse tree (near leaves). **Looser binding** = Evaluated Last = Higher in parse tree (near root).
Example Schema. For grammar rules $S \to aSb \mid \varepsilon$, the yield is the word aab:

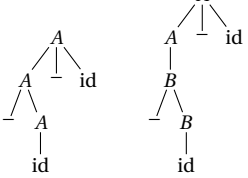


Ambiguity Resolution for $G : A \to -A \mid A - \text{id} \mid \text{id}$. Parse trees for `-id-id`:
Infix priority (G_i)**:** $-(\text{id} - \text{id})$
$G_i : A \to C \mid -A$
$C \to \text{id} \mid C - \text{id}$
Prefix priority (G_p)**:** $(-\text{id}) - \text{id}$
$G_p : A \to B \mid A - \text{id}$
$B \to -B \mid \text{id}$

Ambiguous **Solution** (G_i)



Ambiguous **Solution** (G_p)



$L(G) = L(-^* \text{id}(-\text{id})^*)$
Divisibility check.
For binary string $(b_k b_{k-1} \ldots b_0)$, to check divisibility by integer n:
1. Initialize $r = 0$ 2. Scan bits from left to right, updating $r \leftarrow (2r + b_i) \bmod n$
After the last bit:
- if (r=0), the binary number is divisible by (n)
- otherwise, it is not

Special case. if $(n = 2^m)$, just check whether the last (m) bits are all (0).
L_1**: binary strings divisible by 3**, NFA: **2. binary strings divisible by 4**, NFA:
After init state, state is q_k represents $r = k$

State	0	1
q_{init}	q_0	q_1
◎ q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

State	0	1
q_{init}	q_0	q_1
◎ q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_0	q_1
q_3	q_2	q_3

Example. $e_2 = [+|-]?\Big(([0\text{-}9]^+(.[0\text{-}9]^*)? \mid .[0\text{-}9]^+)([eE][+|-]?[0\text{-}9]^+)?\Big)$

Inside-out:

$$A = [+|-]? \quad \to \text{true} \quad \text{(optional)}$$
$$B = [0\text{-}9]^+ \quad \to \text{false} \quad \text{(one-or-more, base not nullable)}$$
$$C = (.[0\text{-}9]^*)? \quad \to \text{true} \quad \text{(optional)}$$
$$D = .[0\text{-}9]^+ \quad \to \text{false} \wedge \text{false} = \text{false}$$
$$BC \mid D : (\text{false} \wedge \text{true}) \vee \text{false} \quad = \text{false}$$
$$E = ([eE][+|-]?[0\text{-}9]^+)? \quad \to \text{true} \quad \text{(optional)}$$
$$\text{body} = (BC \mid D) \cdot E \quad = \text{false} \wedge \text{true} = \text{false}$$
$$\text{nullable}(e_2) = A \wedge \text{body} \quad = \text{true} \wedge \text{false} = \boxed{\textbf{false}}$$